

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO3:** Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3, BL6).*

Assessment Tool Weightage: 25%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Data Interpretation

Duration: 1hr

1. A Reinforcement Learning agent is trained for multiple episodes and the reward values increase gradually from 10 to 45. Interpret the trend in rewards and defend what indicates about the learning performance and policy improvement of the agent.

Problem Statement

Analyze the performance of a Reinforcement Learning agent whose rewards increase gradually from 10 to 45 over multiple training episodes. The objective is to determine whether the increasing reward indicates improvement in the learned policy.

Problem Solving Steps

1. Define reward values for different episodes.
2. Store the rewards obtained by the agent.
3. Calculate the improvement between initial and final rewards.
4. Plot episode number against reward.
5. Observe the reward trend.
6. Determine whether the increasing reward indicates policy improvement.

Code

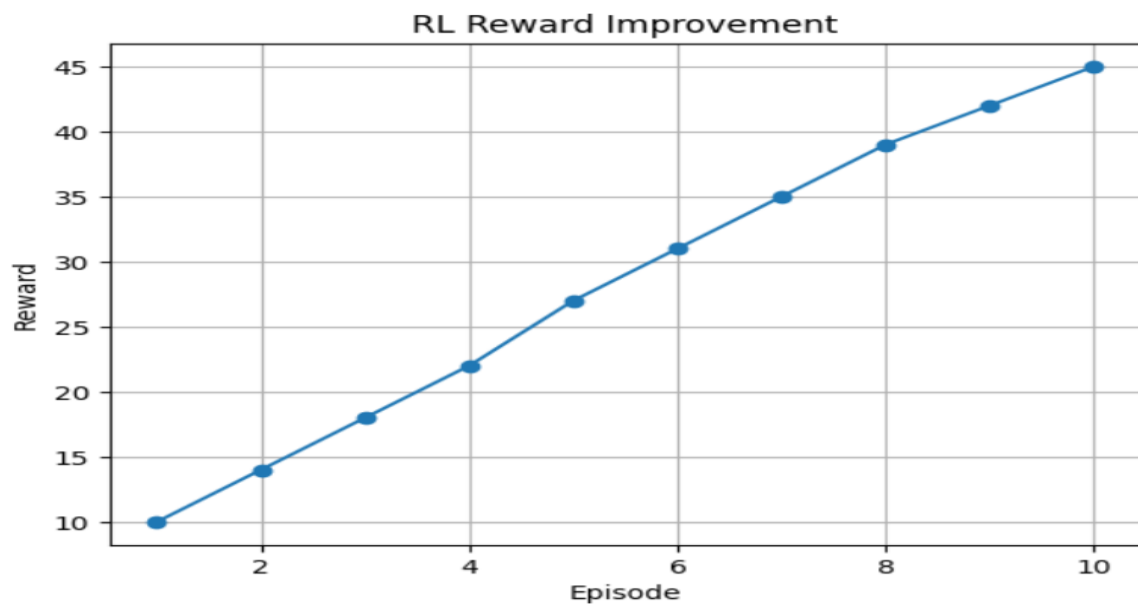
```
import matplotlib.pyplot as plt
episodes = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rewards = [10, 14, 18, 22, 27, 31, 35, 39, 42, 45]
print("REWARD TREND ANALYSIS")
print("-" * 30)
for e, r in zip(episodes, rewards):
    print("Episode:", e, "Reward:", r)
improvement = rewards[-1] - rewards[0]
print("\nInitial Reward:", rewards[0])
print("Final Reward:", rewards[-1])
print("Improvement:", improvement)
plt.plot(episodes, rewards, marker="o")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("RL Reward Improvement")
plt.grid()
plt.show()
```

Output

REWARD TREND ANALYSIS

Episode: 1 Reward: 10
Episode: 2 Reward: 14
Episode: 3 Reward: 18
Episode: 4 Reward: 22
Episode: 5 Reward: 27
Episode: 6 Reward: 31
Episode: 7 Reward: 35
Episode: 8 Reward: 39
Episode: 9 Reward: 42
Episode: 10 Reward: 45

Initial Reward: 10
Final Reward: 45
Improvement: 35



Conclusion

The increasing reward indicates that the agent is learning from its interactions with the environment. The improvement from 10 to 45 shows that the agent is selecting better actions over time. Therefore, the learned policy is improving and becoming more effective. If the reward eventually becomes stable, it can indicate convergence toward a good policy.

2. An epsilon-greedy agent is tested with different epsilon values such as 0.1, 0.3 and 0.5. The cumulative reward obtained is highest when epsilon is 0.1 and lowest when epsilon is 0.5. Justify and apply the concept of exploration vs exploitation to interpret why the reward changes with epsilon and identify which epsilon value provides better performance.

Problem Statement

Analyze the effect of different epsilon values (0.1, 0.3, and 0.5) on the cumulative reward of an epsilon-greedy Reinforcement Learning agent. Determine which epsilon value provides the best performance.

Problem Solving Steps

1. Define different epsilon values.
2. Assign cumulative rewards obtained for each value.
3. Compare exploration and exploitation levels.
4. Plot epsilon values against cumulative rewards.
5. Identify the epsilon value producing the highest reward.
6. Explain the exploration–exploitation trade-off.

Code

```
import matplotlib.pyplot as plt
epsilon = [0.1, 0.3, 0.5]
rewards = [850, 700, 550]
print("EPSILON-GREEDY ANALYSIS")
print("-" * 30)
for e, r in zip(epsilon, rewards):
    print("Epsilon:", e, "Cumulative Reward:", r)
best = rewards.index(max(rewards))
print("\nBest Epsilon:", epsilon[best])
print("Highest Reward:", rewards[best])
plt.bar([str(e) for e in epsilon], rewards)
plt.xlabel("Epsilon Value")
plt.ylabel("Cumulative Reward")
plt.title("Effect of Epsilon on Cumulative Reward")
plt.grid(axis="y")
plt.show()
```

Output

EPSILON-GREEDY ANALYSIS

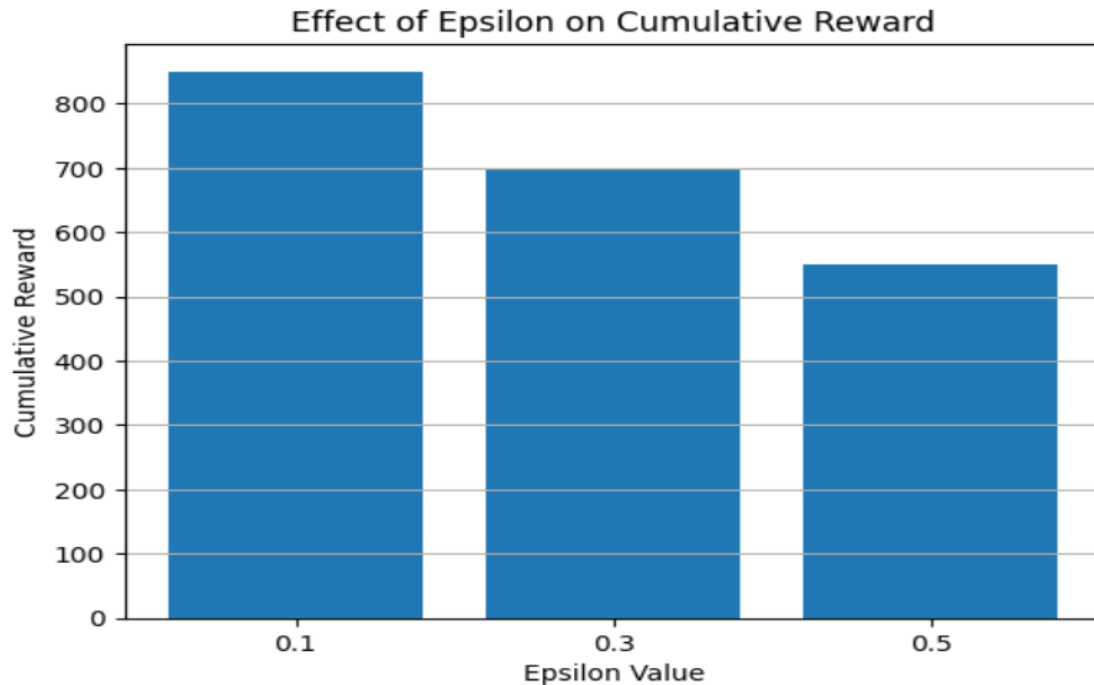
Epsilon: 0.1 Cumulative Reward: 850

Epsilon: 0.3 Cumulative Reward: 700

Epsilon: 0.5 Cumulative Reward: 550

Best Epsilon: 0.1

Highest Reward: 850



Conclusion

An epsilon value of **0.1** provides the highest cumulative reward because the agent spends more time exploiting the best-known actions. Higher epsilon values increase exploration and may cause the agent to select less rewarding actions more frequently. Therefore, $\epsilon = 0.1$ provides the best performance in this experiment.

3. Two Reinforcement Learning algorithms, Q-Learning and SARSA, are compared based on average rewards obtained over 100, 200 and 300 episodes. Q-Learning consistently shows slightly higher reward values than SARSA. Identify the performance difference between these algorithms and determine which algorithm converges faster based on the observed results.

Problem Statement

Compare Q-Learning and SARSA based on their average rewards over 100, 200, and 300 episodes. Determine which algorithm performs better and converges faster based on the observed reward trend.

Problem Solving Steps

1. Define episode checkpoints.
2. Record average rewards for Q-Learning.
3. Record average rewards for SARSA.
4. Plot both reward curves.
5. Compare their performance.
6. Identify the algorithm showing faster improvement and higher rewards.

Code

```
import matplotlib.pyplot as plt
episodes = [100, 200, 300]
q_learning = [60, 78, 90]
sarsa = [55, 72, 84]
print("Q-LEARNING VS SARSA")
print("-" * 30)
for i in range(len(episodes)):
    print("Episodes:", episodes[i])
    print("Q-Learning Reward:", q_learning[i])
    print("SARSA Reward:", sarsa[i])
    print()
print("Final Q-Learning Reward:", q_learning[-1])
print("Final SARSA Reward:", sarsa[-1])
plt.plot(episodes, q_learning, marker="o",
         label="Q-Learning")
plt.plot(episodes, sarsa, marker="s",
         label="SARSA")
plt.xlabel("Episodes")
plt.ylabel("Average Reward")
plt.title("Q-Learning vs SARSA")
plt.legend()
plt.grid()
plt.show()
```

Output

Q-LEARNING VS SARSA

Episodes: 100

Q-Learning Reward: 60

SARSA Reward: 55

Episodes: 200

Q-Learning Reward: 78

SARSA Reward: 72

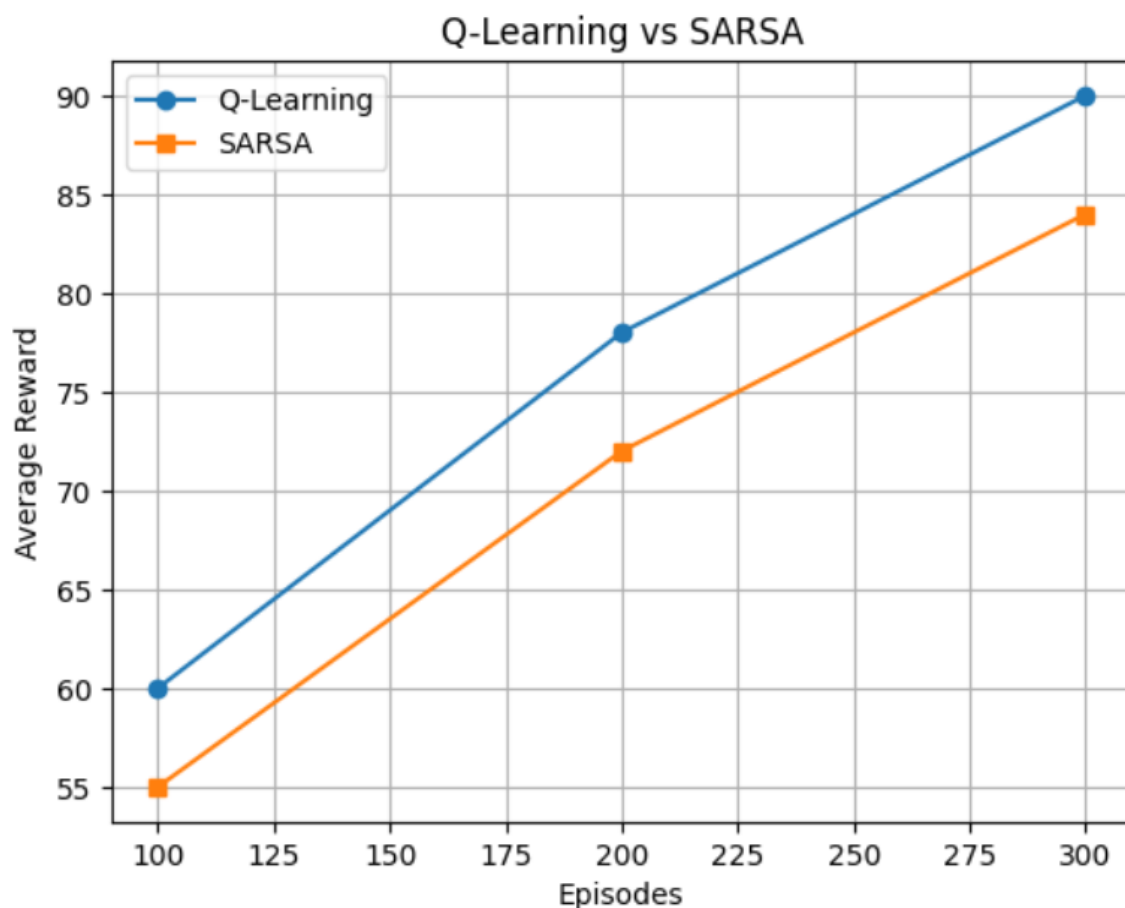
Episodes: 300

Q-Learning Reward: 90

SARSA Reward: 84

Final Q-Learning Reward: 90

Final SARSA Reward: 84



Conclusion

Q-Learning consistently achieves higher average rewards than SARSA at all three checkpoints. The faster increase in reward indicates that Q-Learning learns the optimal action values more effectively in this example. Therefore, Q-Learning demonstrates better performance and faster convergence based on the given observations.

4. An RL agent is trained using different discount factor values such as 0.2, 0.5 and 0.9. The average reward increases as the discount factor increases. Discuss how the discount factor affects long-term reward optimization and justify which discount factor is most suitable for achieving better learning performance.

Problem Statement

Analyze how different discount factor values (0.2, 0.5, and 0.9) affect the average reward of an RL agent. Determine which discount factor provides better long-term learning performance.

Problem Solving Steps

1. Select discount factor values of 0.2, 0.5, and 0.9.
2. Record the corresponding average rewards.
3. Compare the effect of each value.
4. Plot discount factor against average reward.
5. Identify the best-performing discount factor.
6. Explain the importance of future rewards in decision-making.

Code

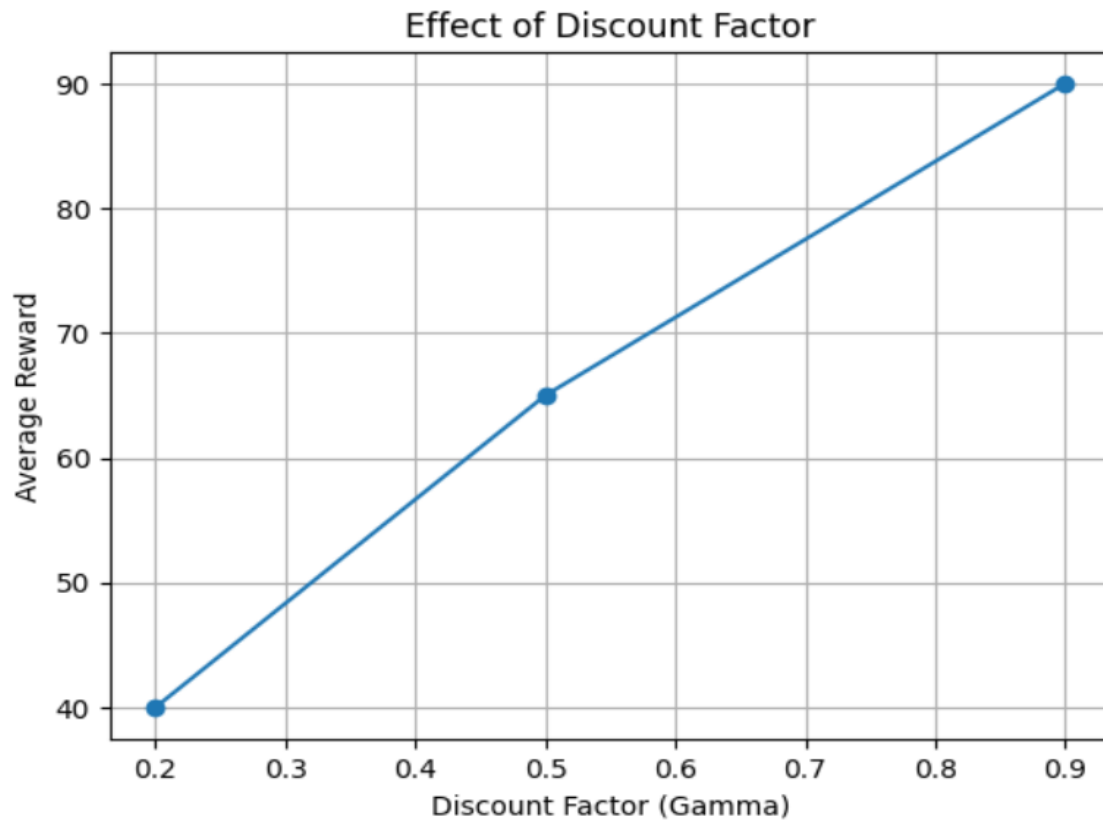
```
import matplotlib.pyplot as plt
gamma = [0.2, 0.5, 0.9]
rewards = [40, 65, 90]
print("DISCOUNT FACTOR ANALYSIS")
print("-" * 30)
for g, r in zip(gamma, rewards):
    print("Discount Factor:", g,
          "Average Reward:", r)
best = rewards.index(max(rewards))
print("\nBest Discount Factor:", gamma[best])
print("Highest Average Reward:", rewards[best])
plt.plot(gamma, rewards, marker="o")
plt.xlabel("Discount Factor (Gamma)")
plt.ylabel("Average Reward")
plt.title("Effect of Discount Factor")
plt.grid()
plt.show()
```

Output

DISCOUNT FACTOR ANALYSIS

Discount Factor: 0.2 Average Reward: 40
Discount Factor: 0.5 Average Reward: 65
Discount Factor: 0.9 Average Reward: 90

Best Discount Factor: 0.9
Highest Average Reward: 90



Conclusion

A higher discount factor gives greater importance to future rewards. In this experiment, $\gamma = 0.9$ produces the highest average reward because the agent considers long-term consequences more strongly. Therefore, 0.9 is the most suitable value among the tested values for achieving better long-term performance.

5. An RL agent is trained in the CartPole environment with different learning rates (0.01, 0.05, 0.1). The agent achieves higher stability at 0.05 compared to the other values. Analyze how learning rate influences convergence and justify which value provides the best trade-off between speed and stability.

Problem Statement

Analyze the effect of different learning rates (0.01, 0.05, and 0.1) on the stability and convergence of an RL agent trained in the CartPole environment. Determine which learning rate provides the best balance between learning speed and stability.

Problem Solving Steps

1. Define three learning rates.
2. Record the average reward obtained for each learning rate.
3. Compare their convergence behavior.
4. Plot learning rate against average reward.
5. Identify the most stable learning rate.
6. Determine the best trade-off between learning speed and stability.

Code

```
import matplotlib.pyplot as plt
learning_rates = [0.01, 0.05, 0.1]
rewards = [180, 245, 210]
print("LEARNING RATE ANALYSIS")
print("-" * 30)
for lr, reward in zip(learning_rates, rewards):
    print("Learning Rate:", lr,
          "Average Reward:", reward)
best = rewards.index(max(rewards))
print("\nBest Learning Rate:",
      learning_rates[best])
print("Highest Average Reward:",
      rewards[best])
plt.plot(learning_rates, rewards,
         marker="o")
plt.xlabel("Learning Rate")
plt.ylabel("Average Reward")
plt.title("Learning Rate vs Performance")
plt.grid()
plt.show()
```

Output

LEARNING RATE ANALYSIS

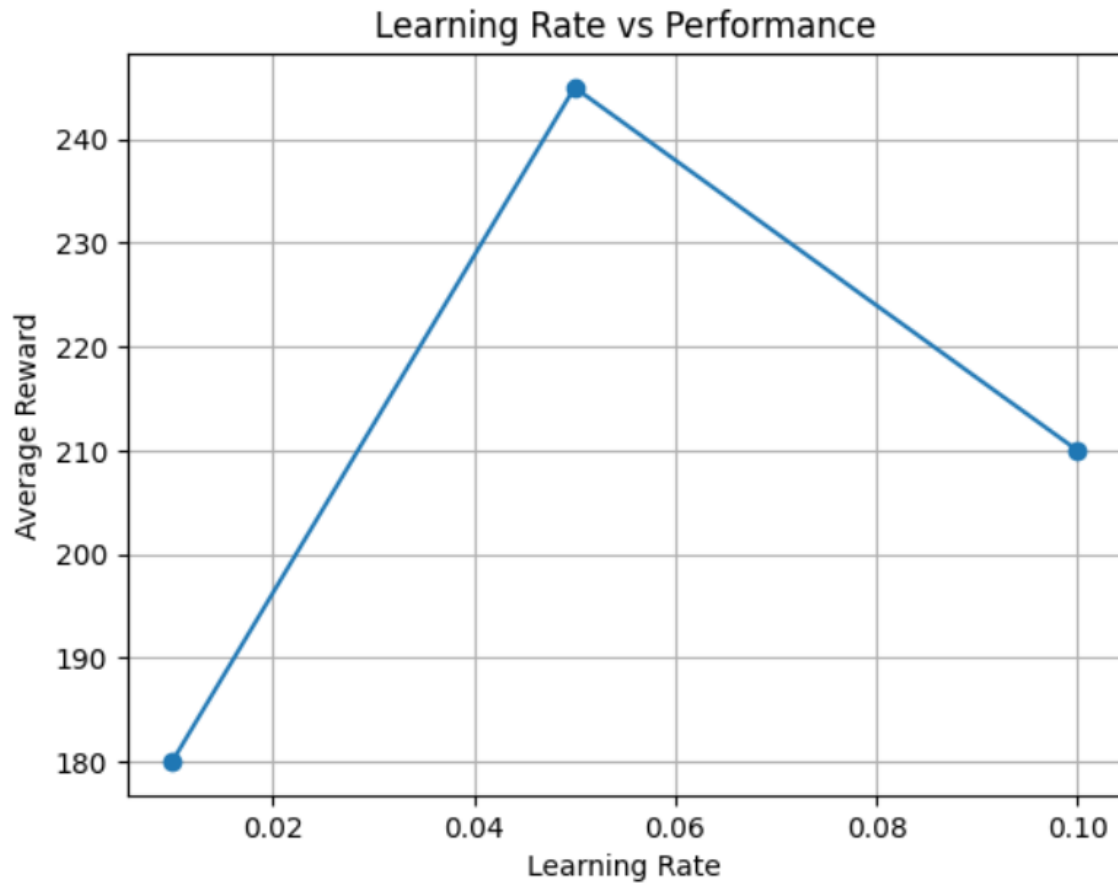
Learning Rate: 0.01 Average Reward: 180

Learning Rate: 0.05 Average Reward: 245

Learning Rate: 0.1 Average Reward: 210

Best Learning Rate: 0.05

Highest Average Reward: 245



Conclusion

A very small learning rate may result in slow learning, while a very high learning rate can cause unstable updates. In this experiment, **0.05** provides the highest average reward and a good balance between learning speed and stability. Therefore, 0.05 is the most suitable learning rate among the tested values.

Code of Conduct:

I S.Sirivally / 192472371 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sirivally.S

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand